

CPSC 250

Memory Allocation and Garbage Collection

Edward Brash

1 Introduction

Memory management is one of the most important concepts in computer science.

Many programming languages require the programmer to manually allocate and free memory.

Python attempts to automate much of this process.

However, understanding what is happening internally is still extremely valuable.

2 Two Philosophies

There are two common approaches to memory management.

2.1 Approach 1

Trust Python completely.

In this approach, we simply allow Python to:

- allocate memory,
- track objects,
- deallocate unused objects.

2.2 Approach 2

Understand the system at a deeper level.

Even though Python automates memory management, understanding the underlying model helps us:

- write better code,
- debug difficult problems,
- understand performance,
- understand why memory leaks occur.

3 The Stack and the Heap

Programs typically use two major memory regions.

3.1 The Stack

The stack is:

- highly ordered,
- fast,
- used for local variables and function calls.

The stack behaves somewhat like a stack of plates.

3.2 The Heap

The heap is:

- less ordered,
- more flexible,
- used for dynamically allocated objects.

The heap behaves more like a pile of objects spread around memory.

4 CPython and the Virtual Machine

Python programs are executed by the Python virtual machine.

The most common implementation is:

CPython

CPython handles much of the memory management automatically.

Languages using similar models include:

- Python,
- Java,
- C#,
- Perl,
- Ruby,
- Lua.

5 All Python Objects Live on the Heap

In CPython, essentially all Python objects are stored on the heap.

This includes:

- integers,
- floats,
- strings,
- lists,
- dictionaries,
- user-defined objects.

6 Reference Counting

CPython keeps track of how many variables refer to each object.

This is called the **reference count**.

When an object is created:

- memory is allocated on the heap,
- the object's reference count is initialized.

7 Garbage Collection

When the reference count of an object falls to zero:

- the object is no longer accessible,
- its memory can be reclaimed.

This process is called garbage collection.

8 Example Class

```
class Test:

    def __init__(self, name):
        self.name = name

    def __del__(self):
        print(f"Deleting {self.name}")
```

The method:

```
__del__()
```

is called when the object is destroyed.

9 Example Program

```
if __name__ == "__main__":  
  
    print("Discarded object creation")  
    Test("hello1")  
  
    print("Saved object creation")  
    my_list = Test("hello2")  
  
    print("Program done")
```

10 What Happens?

10.1 Case 1

```
Test("hello1")
```

creates an object on the heap, but no variable stores a reference to it.
Therefore:

- reference count quickly falls to zero,
- the object becomes garbage collectable almost immediately.

10.2 Case 2

```
my_list = Test("hello2")
```

stores a reference to the object.
Therefore the object remains allocated until the reference disappears.

11 Reference Counting Example with Strings

Consider:

```
s1 = "Python"  
s2 = "Computer Science"  
s3 = s2  
  
s1 = "Zybooks"  
s2 = s1
```

12 Step-by-Step Interpretation

Initially:

```
s1 = "Python"
```

creates a string object on the heap with reference count 1.
Then:

```
s2 = "Computer Science"
```

creates another string object.
Next:

```
s3 = s2
```

causes both `s2` and `s3` to refer to the same object.
The reference count for "Computer Science" increases to 2.

13 Reassignment

Now consider:

```
s1 = "Zybooks"
```

The old string "Python" loses its reference.
If no references remain, its reference count becomes zero and it may be garbage collected.

14 The Garbage Collector Module

Python includes a garbage collection module:

```
import gc
```

We can inspect tracked objects:

```
gc_objects = gc.get_objects()  
  
print(len(gc_objects))
```

This can help analyze memory behavior.

15 Circular References

Reference counting works well most of the time.
However, circular references can cause problems.

16 Dog and Cat Example

Suppose we define:

```
class Dog:  
  
    def __init__(self, name, cousin=None):  
        self.name = name  
        self.cousin = cousin  
  
class Cat:  
  
    def __init__(self, name, cousin=None):  
        self.name = name  
        self.cousin = cousin
```

Now:

```
bailey = Dog("Bailey")
misty = Cat("Misty", bailey)

bailey.cousin = misty
```

creates a circular reference.

17 The Problem

Now:

- Bailey refers to Misty,
- Misty refers to Bailey.

Even if we delete one variable:

```
del bailey
```

the objects may still refer to each other internally.
Their reference counts may never fall to zero.

18 Memory Leaks

This situation can create a memory leak.

A memory leak occurs when memory remains allocated even though the programmer no longer intends to use it.

19 Breaking the Circular Reference

One solution is to manually remove the reference:

```
misty.cousin = None
```

Now the reference cycle is broken.

20 Why This Matters

Programs that repeatedly leak memory may eventually:

- slow down,
- consume large amounts of RAM,
- crash.

21 Key Takeaways

- Python automatically manages most memory allocation.
- CPython uses reference counting.
- Objects live on the heap.
- The stack and heap serve different purposes.
- Objects with zero references may be garbage collected.
- Circular references can cause memory leaks.
- The `gc` module can help inspect memory behavior.